

第 3 课 · 检索与知识底座

混合检索不是「上向量库」四个字就完事

第 3 课 · 120 分钟（讲授 45 + 实操 65 + 小面试 10）

一、这节课的目标

三件事：第一，理解 RAG 全链路的每一环会在哪里丢召回；第二，搞清楚混合检索为什么不是简单地把两个分数加起来；第三，学会用数字证明检索好不好，而不是靠感觉。

代码在 `src/ailab_ops/rag/`，四个模块共约 1000 行。这节课我们会跑 `ailab-ops compare` 拿到真实数字。

二、口语化讲稿

RAG 的失败不是只有「检索不到」

先说一个反直觉的事实：RAG 系统答错，绝大多数时候不是检索器不好，而是流水线上某一环悄悄丢了信息。常见的丢法有四种。

环节	怎么丢的	我们项目里的对应
切分	按段落切，症状和修复方法被切到两块，各自都排不上来	<code>store.py</code> 的分块按段落打包，保留重叠
查询构造	用户问「为什么挂了」，你拿这句话去检索	推理器改成用**观察到的日志原文**去检索

环节	怎么丢的	我们项目里的对应
分词	查询写 memory，文档写 host_mem_used_pct，匹配不上	tokenize 把 snake_case 拆成子词
融合	把词法分和向量分直接相加，量纲不同	RRF 只用排名，或者归一化后再加权

第二行是我最想强调的。****用用户的原话去检索，是 RAG 新手最常见也最致命的错误****。用户问「为什么这个任务挂了」，这句话里没有任何可检索的信息。

正确的做法是：先观察，再检索。我们项目里，推理器会先读日志、读指标，然后把****观察到的具体现象****拼成检索词——比如「AssertionError gpu_util_pct cliff_to_zero」——再拿去检索。这就从「用问题检索」变成了「用证据检索」。

混合检索：不要想当然

接下来是这节课的核心，也是我认为这个项目里最有教学价值的一个发现。

教科书上会说：混合检索 = 词法检索（BM25）+ 向量检索，两者互补，效果更好。听起来很合理。但我们把这个假设实测了一遍，结论是——****在本项目的数据上，朴素地做混合反而比纯词法更差****。

在真实失败日志上对比三种检索器。下面是实测结果。

```
PYTHONPATH=src python3 -m ailab_ops.cli compare
```

检索方式	to p-1 准确率	说明
纯词法 (BM25F)	82.5%	精确错误串的匹配是它的强项
纯向量 (哈希编码器)	29.0%	词袋模型抓不住精确的稀有 token
混合, 等权 (w=0.5)	57.0%	**比纯词法差 25 个点**
混合, w=0.7	67.1%	权重往词法挪, 立刻回升
混合, 偏词法 (w=0.85)	75.2%	接近纯词法
混合, 强偏词法 (w=0.9)	76.6%	本项目当前的默认档位附近

以上 286 条真实失败日志查询上的实测值 (make data 后跑 compare 可复现)。看第二行和第三行的差距：等权融合把准确率从 82.5% 拉到了 57%。一个很差的检索器，把一个很好的检索器的结果投票投掉了。

为什么会这样？因为在这个场景里，查询**就是日志原文**，而精确的错误串恰好是词法检索最擅长的。向量检索在这里几乎没有增量价值。所以正确的做法是：让权重

偏向那个更强的检索器。

我们最终定的默认权重是 0.85。理由是：它离纯词法不远（75% vs 82%），同时保留了向量侧在「人用自己的话描述症状」这类释义查询上的增益。这个数字是**扫出来的**，不是拍的。

这个结论可以推广：**融合不会自动胜过它的组件**。如果你的两个检索器强弱悬殊，等权融合一定变差。面试里能说出这一点的人不多。

为什么向量那侧这么弱

因为我们用的是零依赖的**哈希编码器**（feature hashing），不是真实的 embedding 模型。

- 理由一：**离线可复现**。一个必须先下载模型才能启动的项目，在受限网络环境下跑不起来，而且模型一升级输出就变。
- 理由二：**要讲的课是混合检索本身**——如何结合、如何调权重、向量检索为什么会漏掉精确错误串。哈希编码器完全能承载这些。
- `rag/embed.py` 留了可插拔接口，换成真实模型只改一个类。

但这个设计选择有个诚实说明：哈希编码器的弱，**削弱了混合检索在这个项目里的收益**。换成真实的 embedding 模型之后，向量侧的 top-1 会明显上升，最优权重也会往中间移动。这个我可以告诉你，但你必须自己试。

BM25 里两个容易被忽略的细节

第一，**分词**。我们的 tokenize 会把 `host\_mem\_used\_pct`

同时输出成整串和它的子词

`host`、`mem`、`used`、`pct`。这一个决定带来的召回提升，比调任何 BM25 参数都大。

第二，**字段权重**。文档分三个字段：标题、签名（故障会打印的具体字符串）、正文。权重分别是 3、4、1。为什么签名最重？因为在这个领域里，查询通常是日志原文，而签名字段装的就是日志原文。

src/ailab_ops/rag/bm25.py。为什么 body 权重最低——因为它最长，最容易「顺带提到」某个主题。

```
DEFAULT_FIELD_WEIGHTS = {  
    "title":      3.0,    # runbook 0000000000  
    "signature":  4.0,    # 0000000000000000
```

```
"body":      1.0,   # 000000000000000000000000
}
```

最后那个注释是重点。正文最长的文档，天然更容易命中查询词——哪怕它并不是在讲这件事。给正文低权重、给精确字段高权重，就是在纠正这个偏差。这和 IDF 是同一个思想。

三、动手

实操 1：看检索器内部到底谁投了票

每一行会打印 fused 分数，以及词法排名和向量排名各是多少。

```
PYTHONPATH=src python3 -c "
from ailab_ops.rag import build_knowledge_base
kb = build_knowledge_base()
r = kb.retriever
for q in ['CUDA out of memory',
          'Watchdog caught collective operation timeout',
          'No space left on device ENOSPC']:
    print(r.explain(q, top_k=3)); print()
"
```

看输出的 lex 和 dense 那两列。你会看到有些查询两者一致，有些分歧很大。分歧大的那些，就是融合在起作用的地方。

实操 2：亲手把权重调坏

把 0.5 那行的结果和第 88% 那一行对比，就是等权融合的年代。

```
PYTHONPATH=src python3 -c "
from ailab_ops.rag import build_knowledge_base
for w in [0.0, 0.3, 0.5, 0.7, 0.85, 1.0]:
    r = build_knowledge_base().retriever
    r.lexical_weight = w
    hits = r.search('Watchdog caught collective operation timeout', top_k=1)
    doc = hits.hits[0].doc.metadata.get('scenario')
    print(f'lexical_weight={w:.2f} -> {doc}')
"
```

实操 3：检索好不好，用数字说

这是这节课最重要的技能：**不要用感觉判断检索质量**。我们的测试里有一个就是拿真实失败日志去量 top-1 和 top-3 召回。

真实失败日志上的召回测试。fail 了就是真的变差了。

```
PYTHONPATH=src python3 -m pytest tests/test_rag.py -v -k retrieval
```

顺便注意 tests/test_rag.py::test_fusion_weighting_is_lexically_tilted。它断言的是**结果**（默认权重重要赢过等权），不是权重值本身。这样调权重是自由的，但把效果调坏是不允许的。

四、小面试（10 分钟）

Q1：你的 RAG 知识库数据从哪来？怎么保证质量？

这是把很多人问倒的题。分三层答：

1. **来源**：本项目的知识库是从故障剧本 `faults.yaml` **派生**的——每个场景的 runbook 直接成为一篇文档。好处是数据和质量天然一致。
2. **质量**：每个场景都必须有 remediation 和 runbook body，而且 body 不得少于一定长度，这些由测试强制（test_playbook.py）。
3. **评测**：拿真实失败日志当查询，量 top-1 / top-3 召回和 MRR。检索是不是变好，看数字，不看感觉。

第三点是分水岭。「我做了 RAG」和「我能证明我的 RAG 检索质量是多少」是两回事。

Q2：为什么用 BM25 而不是直接上向量库？

- 在「查询是精确错误串」的场景下，词法检索更强——这是实测结论，不是偏好
- BM25 不需要训练、不需要 GPU、结果完全确定、可解释
- 真正的问题是它怕措辞变化，而措辞变化靠向量那侧补
- 所以答案是**两者都要**，但权重不能等分

Q3：你怎么知道检索器出问题了，而不是模型出问题了？

这是工程上非常实际的问题。答法：「我会把评测拆成两段。先单独量检索的召回率——给定真实的失败日志，看正确的 runbook 有没有被检索到。如果检索召回高但端到端准确率低，那是模型或提示词的问题；如果检索召回本身就低，那先修检索，调提示词是浪费。」

本项目里对应的命令就是 `ailab-ops compare`。

Q4：分块策略怎么选？

- 按段落打包，而不是按固定字数切——本项目的 runbook 是一段一个意思
- 保留重叠，避免症状和修复方法被切开后两边都排不上来
- 块太大：噪声增加、检索变钝；块太小：上下文碎片化、模型接不上
- 最终还是要用召回率来定，而不是靠直觉

五、作业

1. 把 `rag/store.py` 里 `DEFAULT\_LEXICAL\_WEIGHT` 扫一遍 0.0 到 1.0，画出「权重 vs top-1 准确率」的曲线，找出拐点。
2. 写三句**释义查询**（用你自己的话描述某个故障，不要抄日志原文），看纯词法和混合各能不能检索对。这能让你直观感受到向量侧的价值。
3. 思考题：如果数据量放大一万倍（一百万篇文档），我们这套「全量打分」的实现会先在哪里崩？